

S C S 1 3 0 3

Introduction to Software Engineering

Section 7 · Software Testing

Prof. K. P. Hewagamage

B.Sc. Computer Science · Year 1, Semester 1

University of Colombo School of Computing

Section 7 — Intended Learning Outcomes

By the end of this section, students should be able to:

- LO1 Explain what software testing is, state its two main goals, and describe its place in the V&V framework
- LO2 Distinguish between black box and white box testing and apply the correct approach at each testing stage
- LO3 Apply equivalence partitioning and boundary value analysis to design effective test cases
- LO4 Describe unit, component, and system testing — and write automated tests using pytest or JUnit
- LO5 Explain Test-Driven Development and describe the Red-Green-Refactor cycle
- LO6 Describe release testing — requirements-based, scenario, and performance testing
- LO7 Explain alpha, beta, and acceptance testing and how agile projects handle user testing
- LO8 Describe how AI tools assist in test generation and what professional responsibilities engineers retain

Section 7 — Roadmap

7.1

What is Software Testing?

7.2

Black Box and White Box Testing

7.3

Test Design Techniques

7.4

Development Testing

7.5

Test-Driven Development

7.6

Release Testing

7.7

User Testing

7.8

Modern Testing Practices

7.9

AI in Software Testing

What is Software Testing?

The discipline of finding problems before users do

What is Software Testing?

Definition

Software testing is the process of executing a program with the intent of finding defects, and verifying that the program does what it is supposed to do. It is the primary activity by which we gain confidence that software is ready for use.

Two goals that every test activity must serve:

Goal 1: Validation

Show that the system does what it is intended to do.

Test cases are designed to demonstrate correct behaviour — that the system meets its requirements when used as expected.

Goal 2: Defect Discovery

Discover situations where the behaviour is incorrect, undesirable, or does not match its specification.

Test cases are designed to BREAK the system — to force it to fail and reveal hidden defects.

The Fundamental Limit of Testing

Testing can prove the presence of bugs — never their absence

**"Testing can reveal the presence of errors — but not their absence." —
Edsger Dijkstra, 1969**

What this means in practice:

Exhaustive testing is impossible

A simple function with two integer inputs has $2^{64} \times 2^{64}$ possible combinations. No team can run that many tests.

Passing tests $\neq$ correct software

Tests that pass tell you the software works for the cases you thought to test — not for all possible inputs.

Testing finds bugs — not guarantees

After testing, you can say: 'We found no bugs in these scenarios.' You cannot say: 'There are no bugs.'

This is why design principles matter

Good architecture and coding practices reduce defects before testing even begins — testing is the last line of defence, not the first.

Verification and Validation — The Framework

Testing sits inside a broader quality assurance process

Verification

"Are we building the product right?"

Checking that the software conforms to its specification. Internal consistency — does what we built match what was designed?

Methods: code reviews, static analysis, inspections

Validation

"Are we building the right product?"

Checking that the software meets the real needs of its users. The specification itself may be wrong — validation catches this.

Methods: testing with real users, acceptance tests

Two complementary V&V techniques:

Static V&V (Inspections)

Analysis of the code, design, or requirements without running the software. Reviews, walkthroughs, and static analysis tools. Can start before code exists.

Dynamic V&V (Testing)

Executing the software with test data and observing its behaviour. Can only check what you can run — but catches behaviour errors inspections miss.

The Testing Stages — An Overview

Testing happens at multiple stages — each catches different kinds of problems

Testing is not a single activity at the end of development. It happens throughout the lifecycle.

1	Unit Testing <i>Who: Developer</i>	Test individual functions and methods in isolation	 pytest / JUnit / Jest
2	Component / Integration Testing <i>Who: Developer / QA</i>	Test groups of units working together — check interfaces	 Integration test frameworks
3	System Testing <i>Who: QA team</i>	Test the complete integrated system against requirements	 End-to-end test tools
4	Release Testing <i>Who: Separate test team</i>	Validate the release candidate — requirements, scenarios, performance	 Load testing (JMeter, k6)
5	User / Acceptance Testing <i>Who: Customer / Users</i>	Confirm the system meets real user needs in their environment	 Manual + automated

Black Box and White Box Testing

How you approach test design determines everything

Two Fundamental Approaches to Testing

The most important distinction in test design

Why this matters

Understanding whether a test is black box or white box determines what inputs you choose, what you check, and what kinds of defects you can find. You choose the approach before writing a single test case.

Black Box Testing

- **Test based on: SPECIFICATION**

You know what the function is supposed to do — you test whether it does it

- You see: Inputs and outputs only

The internal code is hidden — you cannot see HOW the result is produced

- You design tests from: Requirements, use cases, user stories

Not from reading the source code

- Finds: Missing features, wrong outputs, specification failures

Does not find: unused code, unreachable logic

White Box Testing

- **Test based on: CODE STRUCTURE**

You examine the implementation to design tests

- You see: The actual source code

You can trace exactly what each path through the code does

- You design tests from: Logic branches, conditions, loops

Ensure every path through the code is exercised

- Finds: Logic errors, dead code, uncovered branches

Does not find: features that were never implemented

Black Box vs White Box — Which to Use When

Each testing stage leans on one approach — sometimes both are needed

Testing Stage	Primary Approach	Why
Unit Testing	White Box	Developer knows the code; tests every branch and condition in the function
Component / Integration	Both	Interfaces are tested externally (black); internal logic checked internally (white)
System Testing	Black Box	Tests are derived from requirements — testers do not need to read the code
Release Testing	Black Box	Separate test team uses specifications only — independence is essential
Acceptance Testing	Black Box	Users test what the system does, not how it works internally
Security Testing	Both	External attack testing (black) + code review for vulnerabilities (white)

In practice, professional test teams use both approaches together — black box for specification conformance, white box for thoroughness.

Test Design Techniques

Choosing inputs systematically — not randomly

Why Systematic Test Design Matters

Random testing misses the cases that matter most

The challenge

You cannot test every possible input. There are millions. You must choose a SMALL set of test cases that maximise the chance of finding real defects. Systematic techniques help you make that choice intelligently.

Three techniques you must know:

1

Equivalence Partitioning

Divide inputs into groups that behave the same way. Test one from each group — no need to test every value.

2

Boundary Value Analysis

Defects cluster at the edges of input ranges. Always test the minimum, maximum, and just beyond each boundary.

3

Use-Case Based Testing

Derive test cases from use cases. Each use case tests a realistic user interaction through the complete system.

Equivalence Partitioning

Group inputs that behave the same — test one from each group

Principle

Inputs can be divided into partitions — groups where the program is expected to behave identically for every value in the group. If the program handles one correctly, it handles all correctly. One test per partition is sufficient.

UCSC Student Portal — Course Registration: credit limit per semester is 12–24 credits

Invalid (too low)

- 0 credits
- 5 credits
- 11 credits

*TC-EP-01: credits=5 →
expect REJECTION*

Valid

- 12 credits
- 18 credits
- 24 credits

*TC-EP-02: credits=18 →
expect ACCEPTANCE*

Invalid (too high)

- 25 credits
- 30 credits
- 40 credits

*TC-EP-03: credits=30 →
expect REJECTION*

Key rule: test one value from each partition. Testing many values from the same partition is wasted effort.

Boundary Value Analysis

Defects cluster at the edges — always test the boundaries

Why boundaries matter

Studies show that most programming errors occur at the boundary of valid input ranges — the exact edge values, just below the minimum, and just above the maximum. Always test these points specifically.

UCSC Student Portal — GPA display: shown only when student has 1 or more completed courses

0 courses	Just below minimum	GPA section NOT displayed
1 course	Minimum boundary	GPA section IS displayed
2 courses	Just above minimum	GPA section IS displayed
10 courses	Typical middle value	GPA section IS displayed
Max courses	Maximum boundary	GPA section IS displayed
Max + 1	Just above maximum	Error: exceeded max enrolment

*Rule: always test min-1, min, min+1, max-1, max, max+1.
These six values catch the majority of boundary defects.*

Use-Case Based Testing

Derive tests from real user scenarios — not just input values

Principle

Each use case represents a realistic interaction between a user and the system. Deriving test cases from use cases ensures you test complete, meaningful workflows — not just isolated functions.

UCSC Student Portal — Use Case: 'Student registers for a course'

Test cases derived from this use case:

TC-UC-01	Normal flow	Student with 15 credits selects available CS1101 — expects confirmation and updated timetable	PASS
TC-UC-02	Course full	Student selects CS1101 which has no remaining seats — expects clear 'Course Full' message	PASS
TC-UC-03	Timetable clash	Student selects CS1203 which clashes with already-registered CS1101 — expects clash warning	DEFECT FOUND
TC-UC-04	Prerequisite not met	Student in Year 1 attempts to register for a Year 3 course — expects prerequisite rejection	PASS
TC-UC-05	Credit limit exceeded	Student tries to add a 4-credit course when already at 24 credits — expects credit limit error	PASS

Writing a Test Case — The Structure

Running example: UCSC Student Portal

Every test case must answer these questions clearly

A test case is a complete specification of one test — inputs, expected output, and the result.
Without this structure, testing is not repeatable.

ID	Description	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-LOGI-N-01	Valid login succeeds	User is registered; system is running	1. Open portal login page 2. Enter valid studentID and password 3. Click Sign In	Portal home page displayed; student name shown in header	Portal home displayed	PASS
TC-LOGI-N-02	Wrong password rejected	User is registered	1. Open login page 2. Enter valid studentID, wrong password 3. Click Sign In	Error message: 'Invalid credentials'	Error shown	PASS
TC-LOGI-N-03	Account locked after 5 failures	User account exists	1. Attempt login with wrong password 5 times	Account locked message; login disabled	Account NOT locked	FAIL

Section 7.4

Development Testing

Unit · Component · System — testing during building

Development Testing — Overview

Three levels, run by the development team during construction

What is development testing?

Development testing is all testing carried out by the development team while building the system — before any release to external users. It includes unit, component/integration, and system testing, each at a different level of the system.

Three levels of development testing:

Unit Testing

- 1 Individual functions and methods tested in isolation. The developer writes tests for code they just wrote. Fastest to run — run on every code change.

Component / Integration Testing

- 2 Groups of units tested together. Focus on the INTERFACES between components — does the data pass correctly between modules?

System Testing

- 3 The complete integrated system tested against requirements. Tests the emergent behaviour — properties that only appear when the whole system runs together.

Unit Testing

Testing individual functions in isolation — the foundation of all testing

What is a unit?

A unit is the smallest independently testable piece of software — a single function, method, or class. Unit tests check that each unit does what its specification says, ignoring everything around it.

What makes a good unit test:

Fast

Runs in milliseconds — you run hundreds every time you save a file. Slow tests do not get run.

Isolated

Tests ONE thing. No database, no network, no file system — replace those with test doubles (mocks).

Repeatable

Same result every time. Does not depend on the order tests run or on the state of other tests.

Automated

Runs without human intervention. Integrated into the build so every commit triggers all tests.

Self-checking

The test itself reports pass or fail — no manual inspection of output required.

Unit Testing with pytest — Python

Writing real unit tests for the UCSC Student Portal fee calculation

pytest is the most popular Python testing framework — clean, simple, and powerful. Install with:
`pip install pytest`

Python — pytest unit tests for fee calculation (test_fee_calculator.py)

```
# file: test_fee_calculator.py
import pytest
from fee_calculator import calculate_outstanding_fees

# — Normal cases —
def test_student_with_no_outstanding_fees():
    result = calculate_outstanding_fees(student_id="S001", paid=50000, total=50000)
    assert result["outstanding"] == False
    assert result["amount"] == 0.0

def test_student_with_partial_payment():
    result = calculate_outstanding_fees(student_id="S002", paid=30000, total=50000)
    assert result["outstanding"] == True
    assert result["amount"] == 20000.0

# — Edge cases —
def test_zero_total_fee_raises_error():
    with pytest.raises(ValueError, match="Total fee cannot be zero"):
        calculate_outstanding_fees(student_id="S003", paid=0, total=0)

def test_student_id_not_found_raises_error():
    with pytest.raises(StudentNotFoundError):
        calculate_outstanding_fees(student_id="INVALID", paid=0, total=50000)
```

Run all tests with a single command:

```
pytest test_fee_calculator.py -v
```

Unit Testing with JUnit — Java

The standard Java testing framework — used in enterprise development worldwide

JUnit 5 is the standard unit testing framework for Java. Maven/Gradle projects include it as a test dependency automatically.

Java — JUnit 5 unit tests for FeeCalculator

```
// FeeCalculatorTest.java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class FeeCalculatorTest {

    // — Normal cases —
    @Test
    void studentWithNoOutstandingFees_returnsFalse() {
        FeeResult result = FeeCalculator.calculateOutstanding("S001", 50000, 50000);
        assertFalse(result.isOutstanding());
        assertEquals(0.0, result.getAmount());
    }

    @Test
    void studentWithPartialPayment_returnsCorrectAmount() {
        FeeResult result = FeeCalculator.calculateOutstanding("S002", 30000, 50000);
        assertTrue(result.isOutstanding());
        assertEquals(20000.0, result.getAmount());
    }

    // — Edge cases —
    @Test
    void zeroTotalFee_throwsIllegalArgumentException() {
        assertThrows(IllegalArgumentException.class, () ->
            FeeCalculator.calculateOutstanding("S003", 0, 0));
    }
}
```

*Run: `mvn test` or
`gradle test` →
Report generated
automatically in
`target/surefire-reports/`*

Test Coverage — How Much of Your Code Is Tested?

A metric for evaluating the quality of your test suite

What is test coverage?

Test coverage measures what percentage of your code is actually executed when you run your test suite. High coverage means most of your code is being tested. Low coverage means large areas of code have never been run by a test.

Three types of coverage — each progressively stronger:

Line Coverage

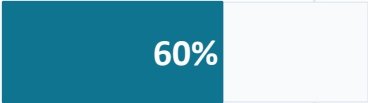
Every LINE of code is executed by at least one test. The most basic metric.
80%+ is typically required in professional projects.

A horizontal progress bar for Line Coverage. It consists of a green segment representing 75% and a light gray segment representing the remaining 25%.

75%

Branch Coverage

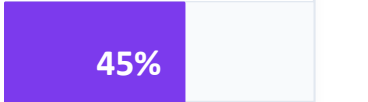
Every BRANCH of every decision (every if-true AND if-false path) is tested.
Stronger than line coverage.

A horizontal progress bar for Branch Coverage. It consists of a teal segment representing 60% and a light gray segment representing the remaining 40%.

60%

Condition Coverage

Every individual condition in each decision is tested as both true and false.
Most thorough — hardest to achieve.

A horizontal progress bar for Condition Coverage. It consists of a purple segment representing 45% and a light gray segment representing the remaining 55%.

45%

Important: 100% coverage does NOT mean zero bugs. Coverage shows what was executed — not whether it was checked correctly. Always pair coverage with good assertions.

Component and Integration Testing

Testing that modules work together correctly

What is integration testing?

Integration testing checks that separately developed components work correctly when combined. The defects it finds are not inside individual units — they are at the INTERFACES between them: wrong data formats, timing issues, incorrect assumptions.

UCSC Student Portal — interface between Registration and Finance subsystems:

Registration Subsystem

`checkFeeStatus(studentID)`

→ calls Finance via interface

receives: {paid:bool,
amount:float}

uses result to allow/deny
course registration

Interface call



← Response

Finance Subsystem

receives: studentID

looks up payment records

returns: {paid,
outstanding_amount}
or raises:

`StudentNotFoundError`

Interface tests to write:

- Paid student → registration allowed
- Unpaid student → registration blocked
- Invalid studentID → error handled gracefully
- Finance subsystem offline → timeout handled

Running example: UCSC Student Portal

Test-Driven Development

Write the test first — then make it pass

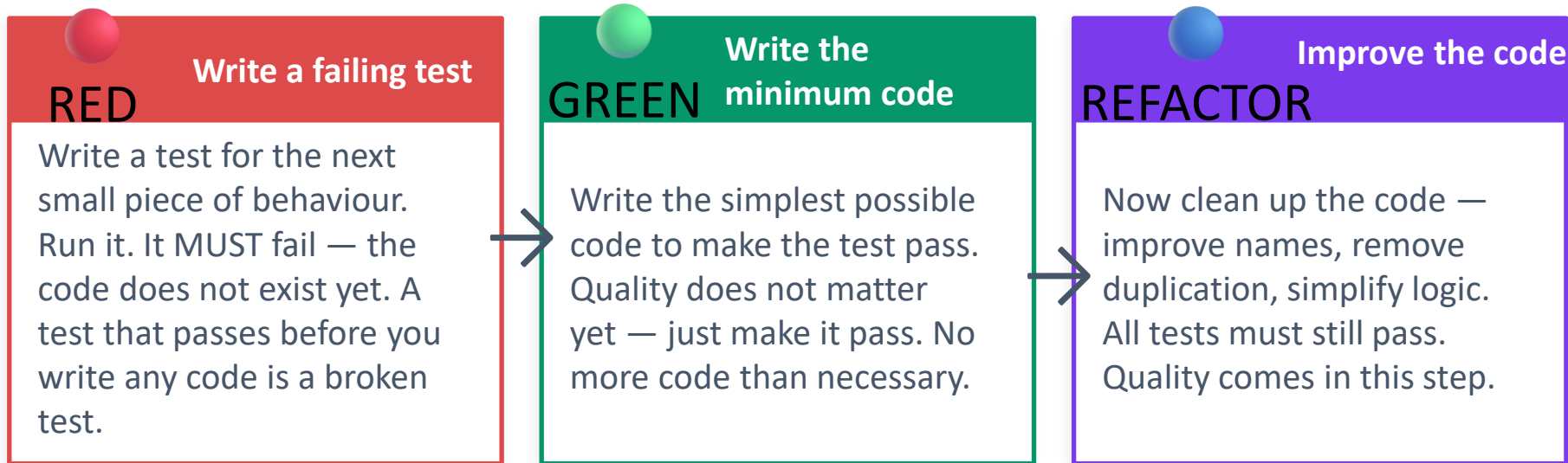
What is Test-Driven Development?

A development discipline — not just a testing technique

Definition

Test-Driven Development (TDD) is a development practice where you write a failing test BEFORE writing the code it is supposed to test. You then write the minimum code to make the test pass. You never write production code without a failing test to justify it.

TDD's three-step cycle — Red, Green, Refactor:



TDD in Practice — UCSC Student Portal

Running example: UCSC Student Portal

Following the Red-Green-Refactor cycle for a real feature

Feature: `calculate_gpa(grades, credits)` — must return credit-weighted GPA

 RED — write failing tests first

```
# STEP 1 — RED: write test first
# Run this → it FAILS (function doesn't exist)
def test_gpa_single_course():
    result = calculate_gpa([3.7], [3])
    assert result == 3.7

def test_gpa_two_courses_weighted():
    # 4.0 (3 credits) + 3.0 (1 credit)
    # = (12.0 + 3.0) / 4 = 3.75
    result = calculate_gpa([4.0, 3.0], [3, 1])
    assert result == 3.75

def test_empty_grades_raises_error():
    with pytest.raises(ValueError):
        calculate_gpa([], [])

# Run: pytest → 3 FAILED (ImportError)
# ✓ RED — tests fail as expected
```

 GREEN then  REFACTOR

```
# STEP 2 — GREEN: minimum code to pass
def calculate_gpa(grades, credits):
    if not grades:
        raise ValueError("No grades provided")
    weighted_sum = sum(g*c for g,c
                       in zip(grades, credits))
    total_credits = sum(credits)
    return weighted_sum / total_credits
# Run: pytest → 3 PASSED ✓
# ✓ GREEN — tests pass
# STEP 3 — REFACTOR: improve without breaking
def calculate_gpa(grades: list[float],
                  credits: list[int]) -> float:
    """Credit-weighted GPA calculation."""
    if not grades or not credits:
        raise ValueError("Grades and credits required")
    if len(grades) != len(credits):
        raise ValueError("Length mismatch")
    weighted = sum(g*c for g,c
                   in zip(grades, credits))
    return round(weighted / sum(credits), 2)

# Run: pytest → 3 PASSED ✓ Code is now clean
```

Benefits of TDD

Why writing tests first leads to better software

1

Code coverage

Every piece of production code has at least one test by construction. You cannot write code without a test first.

2

Fewer defects

Defects are caught within seconds of being introduced — not days or weeks later. The cost of fixing them is almost zero.

3

Better design

Writing tests first forces you to design code that is testable — which means modular, loosely coupled, and with clear interfaces.

4

Living documentation

The test suite documents exactly what every function is supposed to do — in a form that is always current and always executable.

5

Regression protection

Every new feature adds new tests. You can change existing code confidently — if you break something, the tests tell you immediately.

Regression Testing

Protecting what already works when you make changes

What is regression testing?

Regression testing means re-running all existing tests every time you make a change to the code — to verify that the change has not broken anything that was working before. 'Regression' means going backwards.

Why regression testing must be automated:

Manual regression testing is expensive

If the portal has 500 test cases and you fix one bug, running 500 tests manually takes days. Nobody does it — bugs slip through.

Automated regression runs in minutes

The same 500 automated tests run in 2 minutes on every git push. Every defect introduced is caught before it reaches anyone else.

The TDD connection

TDD naturally builds a regression test suite. Every test you write becomes part of the growing safety net that protects all future changes.

Release Testing

Validating a system before it reaches users

What is Release Testing?

A separate team, a complete system, a higher standard

Definition

Release testing is the process of testing a complete version of the system that is intended for use outside the development team. The goal is to convince developers, customers, and the organisation that the system is good enough to release.

Who performs it?:

A SEPARATE test team with no involvement in the original development. Independence matters — familiarity with the code creates blind spots.

What is its goal?:

Not to find every bug — but to validate that the system meets its requirements, performs acceptably, and does not fail during normal use.

How does it differ from system testing?:

“System testing” finds bugs. Release testing verifies the system is ready for external use — validation, not defect hunting.

What does it produce?:

A release decision: approve for deployment, conditional approval with known issues documented, or reject for critical rework.

Requirements-Based Testing

Running example: UCSC Student Portal

Trace every requirement to at least one test

Principle

For every requirement in the SRS or product backlog, there must be at least one test case that verifies it is correctly implemented.

No requirement should be untested at release.

UCSC Student Portal — Sample requirements and their release tests:

REQ-001

The system shall display a warning when a student's outstanding fees exceed LKR 5,000

*TC-RT-001A: Student S001 has LKR 6,000 outstanding
→ warning badge appears on dashboard
TC-RT-001B: Student S002 has LKR 3,000 outstanding
→ no warning badge displayed*

REQ-002

Only registered students shall be permitted to register for courses in the current semester

*TC-RT-002A: Valid active student S001 → registration form accessible
TC-RT-002B: Deregistered student S099 → registration form blocked with explanation*

REQ-003

Course registration shall be closed automatically 14 days after semester start

*TC-RT-003A: System date set to day 15 → registration link not available
TC-RT-003B: System date set to day 13 → registration link available*

Performance and Load Testing

Does the system work well enough under real-world conditions?

Why performance testing is part of release testing

A system can meet all functional requirements and still fail in production because it is too slow, crashes under load, or degrades unacceptably when many users access it simultaneously. Performance must be tested before release.

Types of performance testing:

Load testing

Test under the expected number of concurrent users. For the Student Portal: test with 5,000 students logging in on exam results day.

Stress testing

Increase load BEYOND normal capacity until the system fails. Goal: understand how the system breaks — gracefully or catastrophically?

Spike testing

Simulate sudden peaks — e.g., all students attempting to register when registration opens at 9:00 AM Monday morning.

Endurance testing

Run the system under normal load for an extended period. Reveals memory leaks and resource exhaustion that only appear over hours or days.

Tools: Apache JMeter (free, widely used) · k6 (modern, script-based) · Locust (Python) · Gatling (Scala) · artillery (Node.js)

Section 7.7

User Testing

Real users in their real environment

Why User Testing is Essential

No amount of system testing replaces testing with actual users

The reason

The user's working environment has a major effect on reliability, performance, usability, and robustness — effects that cannot be replicated in a lab. Real users do unexpected things. Real devices, networks, and workflows reveal defects that controlled test environments never surface.

Three types of user testing:

Alpha Testing

The development team invites selected real users to test the software at the developer's site — with the team present to observe.

Goal: find usability and real-world behaviour problems the development team could not see because they know the system too well.

Beta Testing

A pre-release version is made available to a larger group of real users in their own environment — without the development team present.

Goal: discover problems that only emerge in diverse real-world conditions, networks, devices, and usage patterns.

Acceptance Testing: *The customer formally tests the system against their requirements to decide whether to accept delivery from the developer or request further changes.*

Acceptance Testing — The Formal Gate

The customer's right to say: is this what we paid for?

What is acceptance testing?

Acceptance testing is a formal process where the customer tests the delivered system against the agreed acceptance criteria. The outcome is a binary decision: accept delivery or reject it.

The acceptance testing process — six stages:

- | | | |
|---|-----------------------------------|---|
| 1 | Define acceptance criteria | Before the project starts — what must the system do to be accepted?
Written into the contract or user story acceptance criteria |
| 2 | Plan acceptance testing | Who will test? When? What test data will be used? What counts as pass? What is the escalation path if something fails? |
| 3 | Derive acceptance tests | Tests are derived from the acceptance criteria — one test per criterion.
Customer signs off on the test plan before testing begins |
| 4 | Run acceptance tests | Customer runs the tests in their environment, with their data, under their normal working conditions |
| 5 | Negotiate results | Minor failures may be accepted with a fix schedule. Major failures result in rejection. Disputes are resolved against the original criteria |
| 6 | Accept or reject | Formal written acceptance or rejection is issued. Acceptance triggers final payment. Rejection triggers another development cycle |

In Agile: the customer is part of the team and accepts each user story at Sprint Review.

Acceptance is continuous, not a single final gate.

Section 7.8

Modern Testing Practices

How professional teams test software in 2026

Continuous Testing in CI/CD

Testing that happens automatically on every code change

The connection to Section 6

Every git push triggers GitHub Actions. The automated steps that run are primarily TESTS. Continuous testing means your test suite runs without human action — on every commit, every pull request, and every merge.

The CI testing pipeline — what runs automatically on every push:

- | | | | |
|---|-------------------|------|--|
| 1 | Linting | ~5s | Code style and syntax errors — fail fast if code quality rules are broken |
| 2 | Unit tests | ~30s | All unit tests run — any failure blocks the pipeline immediately |
| 3 | Integration tests | ~2m | Component interface tests run against a test database and test API |
| 4 | Coverage report | ~10s | Coverage drops below threshold (e.g. 80%) → pipeline fails |
| 5 | Security scan | ~1m | SAST (Static Application Security Testing) — scans code for known vulnerability patterns |
| 6 | Build and package | ~2m | Only if all above pass — package the application for deployment |

API Testing with Postman

Testing HTTP endpoints — the backbone of modern web services

Why API testing matters

Modern software systems are built from web services and APIs. Testing the API directly — independent of the user interface — lets you verify that the server-side logic is correct before the front-end is even built.

UCSC Student Portal API — testing the course registration endpoint:

Postman tests for the Course Registration API (JavaScript test runner)

```
pm.expect(body.courseCode).to.equal("CS1101");
});
```

Postman Test: POST /api/v1/registration

Request body: { "studentId": "S001", "courseCode": "CS1101" }

— Test 1: successful registration

— Test 2: course already full

pm.test("Status code is 201 Created", () => pm.response.to.have.status(201));

pm.test("Response contains confirmation", () => {

const body = pm.response.json();

pm.expect(body.status).to.equal("confirmed");

pm.test("Full course returns 409 Conflict", () =>

pm.response.to.have.status(409));

pm.test("Error message is meaningful", () => {

pm.expect(pm.response.json().message).to.include("no remaining seats");

});

Tool

Postman

Purpose

GUI tool for designing, sending, and testing API requests
— industry standard

Website

postman.com

REST Assured

Java library for API testing — used inside JUnit test suites

rest-assured.io

pytest + httpx

Python API testing — send HTTP requests inside pytest tests

httpx.encode.co

Mutation Testing — Are Your Tests Good Enough?

A technique for evaluating the quality of a test suite itself

What is mutation testing?

Mutation testing introduces deliberate, tiny defects (mutations) into your source code — then checks whether your test suite detects them. If your tests pass when there is a bug in the code, your tests are not checking thoroughly enough.

How mutation testing works:

Mutations introduced automatically by the tool

Original code

```
def is_fee_overdue(amount, days_late):  
    return amount > 0 and days_late > 30
```

Mutation 1: change > to >=

```
return amount > 0 and days_late >= 30
```

Mutation killed: Test suite detects the bug — the tests are strong enough for this code path

Test suite checking — does it catch the mutations?

Your test suite

```
def test_fee_overdue():  
    assert is_fee_overdue(1000, 31) == True  
    assert is_fee_overdue(0, 45) == False
```

Mutation 1: test catches it? YES ✓ (31 >= 30 still

Mutation survived: Test suite FAILS to detect the bug — you need more test cases here

Tools: PIT (Java), mutmut (Python), Stryker (JavaScript). Aim for >80% mutation score on critical business logic.

Section 7.9

AI in Software Testing

The fastest-changing area of software engineering practice

AI is Transforming Software Testing

From manual test design to AI-generated test suites — in seconds

The opportunity

Writing comprehensive test cases manually is time-consuming and error-prone — engineers often miss edge cases they did not think to consider. AI tools can generate test cases from code or specifications in seconds, covering scenarios human testers commonly overlook.

Where AI helps most in testing today:

Test case generation

Given a function or API specification, AI generates a complete set of test cases — including happy paths, edge cases, and error conditions — faster than a human can.

Code-to-test generation

GitHub Copilot, Diffblue Cover, and similar tools analyse existing source code and automatically write unit tests with assertions. Point at a function — get a test suite.

Bug prediction

AI models trained on large code repositories can predict which code areas are most likely to contain bugs — helping testers focus their manual effort where it matters most.

Anomaly detection

AI monitors running systems and flags behaviour that deviates from the learned normal pattern — detecting bugs in production before users report them.

Visual regression testing

AI compares screenshots before and after a UI change and flags unexpected visual differences — even subtle layout shifts invisible to manual review.

AI Test Generation — A Real Example

Running example: UCSC Student Portal

Generating tests for the UCSC Portal course registration function

Prompt given to GitHub Copilot / Claude:

Prompt to AI

```
"Generate comprehensive pytest tests for register_course(student_id, course_code).
```

AI-generated test suite — must be reviewed before committing

```
# AI-GENERATED test suite (review before using!)
import pytest
from portal.registration import register_course
from portal.exceptions import CourseFullError, TimetableClashError

class TestRegisterCourse:
    def test_successful_registration_returns_confirmation(self, mock_db):
        result = register_course("S001", "CS1101")
        assert result.status == "confirmed"
        assert result.course_code == "CS1101"
```

AI generated 5 solid tests in seconds — including the boundary case a tired developer might miss.

What AI Gets Wrong in Test Generation

Critical awareness — AI-generated tests require professional review

AI-generated tests look comprehensive — but can give false confidence if accepted without review

⚠️ **Tests that always pass**

AI sometimes generates assertions that are always true — assert result is not None — which pass for any input and catch nothing.

⚠️ **Missing business rules**

AI generates tests from code structure. If the code is missing a business rule (e.g. the 5-failed-login lockout from our earlier test case), AI cannot generate a test for it.

⚠️ **Wrong assertions**

AI may assert the wrong expected value — especially for computed results. Always verify that the expected value in the assertion is actually correct.

⚠️ **Mocking done incorrectly**

AI-generated mocks may not accurately simulate real dependencies — the test passes because the mock returns what the test expects, not what the real system would return.

⚠️ **No performance or load tests**

AI generates functional tests. Performance, concurrency, and load testing still require human-designed test plans and specialist tools.

Engineer's Responsibilities in AI-Assisted Testing

What you must always own — regardless of how tests were generated

Define what 'correct' means

AI cannot know your business rules, your regulatory constraints, or your users' expectations. You must define the acceptance criteria that the tests are checking.

Verify the expected values

When AI writes `assert result == 3.75` — verify that 3.75 is actually correct for those inputs. AI can generate syntactically valid but logically wrong assertions.

Identify missing test scenarios

AI generates tests from code patterns. Business-level scenarios — 'what happens when registration opens at 9 AM with 5,000 concurrent students?' — require human insight.

Review tests for completeness

Run mutation testing on AI-generated tests. If mutations survive, the tests are not checking enough — add assertions until the mutation score is acceptable.

Own the test strategy

Test planning — what to test, how thoroughly, at what stage, with what tools — is an engineering decision. AI executes parts of the strategy; you design it.

Keep tests up to date

AI generates tests once. When the code changes, AI-generated tests may pass incorrectly because the mock still returns the old behaviour. Review on every change.

Section 7 — What We Have Covered

Check your understanding against the learning outcomes

LO1

Testing has two goals: validation (correct behaviour) and defect discovery. It sits within the V&V framework, alongside static inspection. Tests prove bugs exist — never their absence.

LO2

Black box tests are designed from specifications — used in system and release testing. White box tests are designed from code — used in unit and integration testing.

LO3

Equivalence partitioning groups inputs that behave identically — test one per group. Boundary value analysis tests the edges of valid ranges where defects cluster.

LO4

Unit tests check individual functions in isolation (pytest / JUnit). Integration tests check interfaces between components. System tests check emergent behaviour. Coverage measures thoroughness.

LO5

TDD: write a failing test (Red), write minimum code to pass (Green), clean up (Refactor). Benefits: coverage, fewer defects, better design, regression protection.

LO6

Release testing: a separate team validates the complete system — requirements-based tests, scenario tests, and performance tests. Load/stress/spike testing precedes deployment.

LO7

Alpha: users test at the developer's site. Beta: pre-release to wider users. Acceptance: customer formally accepts or rejects delivery. In agile, acceptance is continuous at Sprint Review.

LO8

AI generates test cases faster than humans — including edge cases often missed. Engineers must verify expected values, add missing business scenarios, and own the full test strategy.

References and Further Reading

Main Textbooks

Software Engineering

Ian Sommerville, 10th ed., Addison-Wesley, 2015

Chapter 8: Software Testing — primary reading for this section

Software Engineering: A Practitioner's Approach

Roger S. Pressman, 9th ed., McGraw-Hill, 2020

Chapters 22–26: Testing strategies, tactics, and related activities

The Art of Software Testing

Glenford Myers, 3rd ed., Wiley, 2011

The foundational text on systematic test design — highly recommended

Online Resources

- docs.pytest.org — official pytest documentation with tutorials and how-to guides
- junit.org/junit5 — JUnit 5 user guide with examples in Java
- jmeter.apache.org — Apache JMeter performance testing tool (free, open source)
- pitest.org — PIT mutation testing for Java

• postman.com/learning — Postman learning centre for API testing

Coming Up Next

Section 8:

Software Maintenance

Why software changes · Types of maintenance · Managing evolution · Legacy systems · Re-engineering

Tutorial this week:

Write 3 pytest tests for a Student Portal function of your choice · Apply equivalence partitioning to the fee payment system
· Try AI test generation and critique what it produces

Introduction to Software Engineering · Prof. K.
P. Hewagamage